

LizardTech®

MrSID Decode SDK 8 for LiDAR

User Manual

Copyrights

Copyright © 2009–2010 LizardTech. All rights reserved. Information in this document is subject to change without notice. The software described in this document is furnished under a license agreement or nondisclosure agreement. The software may be used or copied only in accordance with the terms of those agreements. No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or any means electronic or mechanical, including photocopying and recording for any purpose other than the purchaser's personal use without the written permission of LizardTech.

LizardTech acknowledges and thanks the many individuals and organizations whose efforts have made our products possible. A full list of copyright, trademark and credit information is available in the document "Copyrights Trademarks and Credits" installed automatically with your product.

LizardTech
1008 Western Avenue, Suite 200
Seattle, Washington, U.S.A. 98104
206.652.5211
www.lizardtech.com

Table of Contents

Chapter 1: Introduction	1
SDK Contents	1
Architecture and Design	3
Chapter 2: Getting Started	5
System Requirements	5
Installation	6
Technical Support	6
Chapter 3: The SDK Classes	9
The PointSource Class	9
The PointWriter Class	10
The Buffer Management Classes	11
The PointIterator Class	12
Text Point Readers and Writers	13
The MG4PointReader Class	13
The Support Classes	13
The Metadata Class	14
Chapter 4: Code Examples	17
Chapter 5: MrSID Decode SDK Command Line Tools	19
Decompressing MG4 Files	19
Viewing File Information	21
Appendix: Company and Product Information	25
About LizardTech	25
Other LizardTech Products	25
MrSID and Patents	27
Index	29

Chapter 1: Introduction

Thank you for using LizardTech® products. This is the documentation for the MrSID® Decode Software Development Kit (SDK) for LiDAR data. The SDK provides a framework for extracting LiDAR data from MrSID Generation 4 (MG4™) files.

LiDAR data is becoming increasingly important to many aspects of business, industry, and government. Because of the enormous quantities of data involved, the use of LiDAR files has been hindered by storage and bandwidth constraints. LizardTech's technologies and products solve these problems and lay the groundwork for truly dynamic LiDAR file access.

Lossless compression with LizardTech LiDAR Compressor enables users to turn giant point cloud data sets into efficient MrSID files that retain 100 percent of the raw data at just 25 percent or less of the original file size (lossless compression). If storage requirements are critical, they can reduce LiDAR file sizes by 90 percent or more by choosing a higher compression ratio and letting LiDAR Compressor select the best way to reach a desired file size (lossy compression).

Used as the foundation for LiDAR Compressor, the MrSID Decode SDK is a robust toolkit suitable for complex application development needs.

NOTE: The MrSID format supports raster data as well as LiDAR data, but a separate set of tools and libraries is used in supporting raster data in the MrSID format. Separate documentation is available in your installation for integrating support for raster-encoded MrSID files.

MrSID Generation 4 (MG4)

The industry standard MrSID format has been trusted as a raster format by geospatial professionals since 1992 and supported in virtually all GIS applications. With the release of LiDAR Compressor LizardTech unveiled a new and improved version of the format, MrSID Generation 4 (MG4). MG4 enables users to view and access their LiDAR data quickly.

SDK Contents

The contents of the MrSID Decode SDK include the following:

Documentation

Cover documentation

The file `README.txt` in the top-level directory and the file `CHANGES.txt` in the `doc` directory contain information about late changes to the SDK.

License

In the top-level directory, the file `LICENSE.pdf` contains the complete licensing information for this SDK.

User Manual

The User Manual (this document) can be found at `doc/UserManual/index.html`.

Reference Manual

The Reference Manual, containing detailed information about each class and method, can be found at `doc/ReferenceManual/index.html`.

Copyrights, Trademarks and Credits

Information about licenses and copyrights, as well as trademark information and acknowledgments, are found in the document `Copyrights_Trademarks_and_Credits.pdf` in the top-level directory.

Headers and Libraries

Headers

The header files for the MrSID Decode SDK are located in subdirectories under the `include` directory. (The Reference Manual provides full documentation for these headers.)

Libraries

The libraries for the SDK are located in the `lib` directory.

Sample Applications

Command-line tools

Several tools are provided in the `bin` directory to aid in development, debugging, and testing. (For information on using these tools, see "Chapter 5: MrSID Decode SDK Command Line Tools" on page 19)

Example Code

A number of example functions are included in the directory `examples/src`. The test files used by these examples are located in `examples/data`. (The Reference Manual provides additional information about these examples.)

NOTE: As a further resource, sample LiDAR images are available for you to work with at http://bin.us.lizardtech.com/lidar/LT_LiDAR_Sample_Data.zip.

Language Bindings

We have added experimental language bindings for C#, Python and Ruby. They are located in the directory `contributions/SWIG`. Please see the `README.txt` in that directory for more details.

Architecture and Design

This section provides an overview of the architecture and some insight into the design philosophy of LizardTech's MrSID Decode SDK.

Basic Terminology

Point – A location in three-dimensional (3D) space with non-optional attributes (X,Y,Z) and optional attributes such as scan angle, pulse intensity, and color.

Channel – All the values for a given attribute. For example, the X channel is all the X values for a given point cloud.

Class Hierarchy

The MrSID Decode SDK is designed around two major classes: objects that are *sources* of LiDAR point data and objects that are *destinations* of LiDAR point data. The sources are derived from the `PointSource` class, and the destinations are derived from the `PointWriter` class. For the SDK, we deal mainly with the `PointSource` class.

The most interesting subclass of the `PointSource` class is `MG4PointReader`. The `PointSource` has two types of methods. The first is for getting properties about the point cloud, and the other type is for accessing the point cloud itself, either the entire cloud or subsets thereof.

Specifying a Region of Interest

When extracting points from the point cloud you must specify the region you wish to extract from, which we call the region of interest. The MrSID Decode SDK uses a bounding box to specify the region of interest.

If you wish to extract all the points in a point cloud, you may do it in either of two ways:

- use the bounding box of the point source
- use `-HUGE_VAL` (-infinity) to `+HUGE_VAL` (+infinity) for the X,Y and Z ranges

Using a bounding box generally defines far more points than a user needs, so when extracting points from a cloud, you must also specify the fraction of the point cloud that you wish to extract. For example if you only want every 20th point, specify 0.05 (1/20) as the fraction value. Use 1.0 when you want all the points.

Point Cloud Data Buffers

When extracting points we use the `PointData` class to pass around parts of the point cloud among functions. This class is a group of channel buffers for the channels that are to be extracted (see "The Buffer Management Classes" on page 11).

Programming and Memory Model

The MrSID Decode SDK separates object allocation and object initialization. This means the constructors do not take arguments and classes have one or more `init()` methods. This makes it easier to work with exceptions and to chain object constructors.

The SDK chooses to use reference counting for objects with long or unknown life spans. The base class for reference counting is `Object`. Its methods, `Object::retain()` and `Object::~release()` increment and decrement the reference counter. Functions and methods that start with “create” create a new reference counted object with a count of one. It is the responsibility of the “create” caller to release the object when done with it using `Object::release()`. When you retain a pointer to an `Object` you must retain the object using `Object::retain()`, until that pointer goes out of scope, at which time you must release the object.

See http://en.wikipedia.org/wiki/Reference_counting for more information on reference counting.

NOTE: The SDKs naming conventions are patterned after those in Objective C.

Thread Safety

The MrSID Decode SDK is thread safe. Once the `PointSource` has been initialized any number of threads can use the `PointSource` instance. The stateful (thread-specific) information for the point extraction is stored in the `PointIterator` class.

Floating Point Quantization

Quantization is a way to convert floating point values to integer values. This facilitates lossless wavelet compression of LiDAR data. LAS files, which contain floating point values, are quantized as part of their storage. The MG4 format quantizes because it uses an integer wavelet transform to achieve lossless compression. The result is that, even with the quantization, LAS files can be compressed losslessly in MG4.

If you are doing any error analysis you must factor the quantization scale into the error bound calculation.

Quantization in the MrSID Decode SDK uses the following conversions between floating point and integer space:

$$\begin{aligned}\langle \text{floating point value} \rangle &= \text{scale} * \langle \text{integer value} \rangle + \text{offset} \\ \langle \text{index value} \rangle &= \text{floor}((\langle \text{floating point value} \rangle - \text{offset}) / \text{scale} + 0.5)\end{aligned}$$

In going from integer space to floating point space and back again using the above methods, the index space values do not change. This stability minimizes the conversion error.

Chapter 2: Getting Started

This chapter provides some preliminary information to get you started using the MrSID Decode SDK. The code examples (see "Chapter 4: Code Examples" on page 17) should give you enough information to determine what level of SDK support your own application will require.

System Requirements

The MrSID Decode SDK is a set of C++ libraries that must be used in conjunction with the specific development environment for your platform. The supported configurations are listed below.

For SDK usage, LizardTech recommends the use of hardware that is at least the equivalent of a 1GHz PC processor with 1 GB of RAM.

NOTE: Each SDK distribution is supported on only a single target platform. Please contact LizardTech for additional distributions for other platforms.

Windows

Windows / Visual Studio 2008 / 32-bit

- built with: Windows 2003/64, VC9.0 – 32-bit
- supported on: Windows 2000/32-bit, Windows 2k3/32-bit, Windows XP/32-bit, Windows Vista/32-bit

Windows / Visual Studio 2008 / 64-bit

- built with: Windows 2003/64, VC9.0 – 64-bit
- supported on: Windows 2003/64-bit, Windows XP/64-bit, Windows Vista/64-bit

Windows / Visual Studio 2010 / 32-bit

- built with: Windows 2003/64, VC10.0 – 32-bit
- supported on: Windows 2003/64-bit, Windows XP/64-bit, Windows Vista/64-bit, Windows 7/64-bit

Windows / Visual Studio 2010 / 64-bit

- built with: Windows 2003/64, VC10.0 – 64-bit
- supported on: Windows 2003/64-bit, Windows XP/64-bit, Windows Vista/64-bit, Windows 7/64-bit

NOTE: The 32-bit version of the SDK can be run on a 64-bit system.

Linux

Linux / Red Hat 4 / 32-bit

- built with: RHEL4/32, gcc 3.4 – 32-bit
- supported on: RHEL4/32

- also supports RHEL5/32 and RHEL5/64, with libstdc++ compatibility library

Linux / Red Hat 4 / 64-bit

- built with: RHEL4/64, gcc 3.4 – 64-bit
- supported on: RHEL4/64
- also supports RHEL5/32 and RHEL5/64, with libstdc++ compatibility library

Linux / Red Hat 5 / 32-bit

- built with: RHEL5/32, gcc 4.1 – 32-bit
- supported on: RHEL5/32

Linux / Red Hat 5 / 64-bit

- built with: RHEL5/64, gcc 4.1 – 64-bit
- supported on: RHEL5/64

Macintosh

Mac OS 10 / Universal / Darwin 9 / 32-bit

- built with: Mac OS 10.5, gcc 4.0, Universal (32-bit)
- supports: Mac OS 10.5 Universal (32-bit) and greater

Mac OS 10 / Universal / Darwin 9 / 64-bit

- built with: Mac OS 10.5, gcc 4.0, Universal (64-bit)
- supports: Mac OS 10.5 Universal (64-bit) and greater

Mac OS 10.6 / Universal / Darwin 10 / 32-bit

- built with: Mac OS 10.6, gcc 4.2, Universal (32-bit)
- supports: Mac OS 10.6 Universal (32-bit)

Mac OS 10.6 / Universal / Darwin 10 / 64-bit

- built with: Mac OS 10.6, gcc 4.2, Universal (64-bit)
- supports: Mac OS 10.6 Universal (64-bit)

Installation

No specific installation is required to use the MrSID Decode SDK beyond copying the SDK contents from the media provided (CD, ISO CD image, archive from FTP site, etc.) to your local computer.

Technical Support

Most technical issues can be resolved using the various resources you have available. In addition to the product documentation and the README file, LizardTech offers a knowledge base and product updates on the LizardTech website.

Knowledge Base

<http://www.lizardtech.com/support/kb/>

The LizardTech Knowledge Base contains articles about known technical and usage issues and is frequently updated.

Developer Website

<http://developer.lizardtech.com>

The LizardTech Developer Website provides you with the tools you need to support viewing MrSID format within your application: downloadable SDKs, technical notes and documentation and a link to additional email support.

Community Forums

<http://www.lizardtech.com/forums/>

The forums are a place to engage in intelligent discourse with the geospatial community. Ask questions, provide answers, and share product usage tips with other Lizardtech customers around the world.

Product Updates

<http://www.lizardtech.com/products>

Updated versions of LizardTech viewer tools are available for download at no cost.

Support Plans

<http://www.lizardtech.com/purchase/other.php>

Protect your investment in LizardTech software by participating in a LizardTech support plan. For more details, please contact your regional LizardTech office.

Contacting Technical Support

<http://www.lizardtech.com/support>

To contact technical support, visit the website at the above URL and follow links to the LizardTech Knowledge Base or the Product Activation page. A Contact Form is also provided for issues that require further assistance.

In an emergency, call 206-902-2845 between the hours of 8 AM and 5 PM Pacific Time.

IMPORTANT: Please have the following information available to assist in resolving your problem:

- Which version of the MrSID Decode SDK you are running
- Other LizardTech products you have installed
- Which operating system you use
- How much free hard drive space your computer has
- How much RAM your computer has

- Version of compiler
- Copy of source code demonstrating the problem, simplified as much as possible
- Relevant test data to allow us to reproduce the problem
- Copy of compiler error messages if appropriate

Chapter 3: The SDK Classes

This chapter describes the important classes of the MrSID Decode SDK.

The PointSource Class

The `PointSource` class is the root class for accessing LiDAR data. Following is a description of each of the methods.

Methods for Accessing Properties

Number of Points

To access the number of points, call `PointSource::getNumPoints()`.

Channels

A channel is all the values for a given attribute. For example, the X channel is all the X values for a given point cloud.

To access the channel information, call `PointSource::getNumChannels()` and `PointSource::getPointInfo()`.

For more information about channels, see "The ChannelInfo Class" on page 11 and "The PointInfo Class" on page 12.

Quantization

If you call the functions `PointSource::getScale()` and `PointSource::getOffset()` and `NULL` is returned, the file is not quantized. Otherwise it returns an array of `doubles` representing the quantization scale and offset values for the X, Y and Z channels.

Even when the LiDAR file is quantized the X, Y, Z value that are extracted from the point cloud are floating point values, not the integer indexes.

For more information about quantization, see "Floating Point Quantization" on page 4.

Metadata

Metadata is auxiliary information about the point cloud stored as key-value pairs. Metadata can be any information the user wishes to add. You can store strings, arrays of floating point values and raw binary data.

To access the channel information, call `PointSource::loadMetadata()`.

For more information see "The Metadata Class" on page 14.

Classification Names

The MrSID Decode SDK stores the classification name as an array of strings. You can use the `ClassId` channel to index into the `ClassIdName` array.

To access the classification names, call `PointSource::getNumClassIdNames()` and `PointSource::getClassIdNames()`.

Methods for Accessing the Point Cloud

Using a Point Iterator

A point iterator is an iterator that gets points for a given bounds.

The function `PointSource::createIterator()` returns an iterator for a given bounds, fraction and set of channels (see "Specifying a Region of Interest" on page 3).

To extract the points, call the function `PointIterator::getNextPoints()`, which walks the specified region of the point cloud until there are no more points to extract (for an example, see "Chapter 4: Code Examples" on page 17).

For more information see "The PointIterator Class" on page 12.

Using Bounds and a Fixed Number of Points

Using bounds and a fixed number of points to extract is much simpler but less versatile.

`PointSource::read()` fills a `PointData` object with the points that most uniformly represent the specified region of interest (for an example, see "Chapter 4: Code Examples" on page 17).

The PointWriter Class

The `PointWriter` class is the base class for writing LiDAR data to files. Following is a description of each of the methods.

Methods for Setting Up and Writing the Output File

Metadata

By default the writers do not copy in the metadata from the point source. It is the responsibility of the application to retrieve the metadata from the source, modify it as necessary and then pass it to the writer using `PointWriter::setMetadata()`. You can also retrieve the metadata for viewing by calling `PointWriter::getMetadata()`,

Quantization

LAS and MG4 files require quantization. By default the writer uses the same quantization as the input point source (see "Quantization" on page 9). However, you can override that behavior by setting quantization explicitly using `PointWriter::setQuantization()`. (For more information about quantization, see "Floating Point Quantization" on page 4).

To access the quantization of the output file, you can use `PointWriter::getScale()` and `PointWriter::getOffset()`.

NOTE: These functions will return `NULL` if the input data is not quantized.

Writing the File

To write the output file, call `PointWriter::write()`. This function writes to a file the point cloud for a given bounds, fraction and set of channels (see "Specifying a Region of Interest" on page 3).

The Buffer Management Classes

When extracting points we use the `PointData` class to pass around parts of the point cloud among functions. This class is a group of `ChannelData` classes for the channels that are to be extracted.

The ChannelInfo Class

A channel is all the values for a given attribute. For example, the X channel is all the X values for a given point cloud.

The three aspects of a channel are:

- name
- data type (floating point, signed integer, etc., stored as a `DataType` enum)
- number of bits of precision

NOTE: For floating point data types, the number of bits of precision is the number of bits you need to store the quantized point value as an integer (for more information, see "Floating Point Quantization" on page 4).

The MrSID Decode SDK handles the following channels:

Standard Channels

<code>X, Y and Z</code>	(Required) The X, Y and Z values specify the physical location of the point.
<code>Intensity</code>	Intensity is the integer representation of the pulse return magnitude.
<code>ReturnNum</code>	The return number is a number that uniquely and sequentially identifies each return from a given output pulse.
<code>NumReturns</code>	The number of returns is the total number of returns from an output pulse.
<code>ScanDir</code>	The scan direction is the direction at which the scanner mirror was traveling at the time of the output pulse.
<code>EdgeFlightLine</code>	The edge of flight line value is the last point on a given scan line before it changes direction. The edge of flight line has a value of 1 only when the point is at the end of a scan (when the mirror is not moving).
<code>ClassId</code>	The classification identifier is an index into the <code>ClassIdName</code> array of the <code>PointSource</code>

instance

Continued >

ScanAngle	The scan angle is an integer representation of the angle off of nadir at which the pulse was output. Negative scan angle value represents an angle to the port side of the plane, and a positive scan angle value represents an angle to the starboard side. Zero is nadir.
UserData	The user data value is any integer value the user wishes to add.
SourceId	The point source identifier identifies a file as the original source of the data.
GPSTime	The GPS time value is the time at which a given point was sampled.
Red, Green, Blue	The red, green and blue values represent the color of the point.
<UserDefinedChannelName>	This channel can be used by the creator of the file for any additional data they would like to include.

The PointInfo Class

The `PointInfo` class is a group of `ChannelInfo` classes that are used to store information about all the channels of a point source. This class is used to obviate passing around arrays of `ChannelInfo` objects and the associated array lengths, which was required when using `PointSource::createIterator()` and `PointData::init()`.

The ChannelData Class

The `ChannelData` class is a derived class of `ChannelInfo` which adds a data buffer and length.

The PointData Class

The `PointData` class is a group of `ChannelData` classes that are used in the point extraction calls `PointIterator::getNextPoints()` and `PointSource::read()`.

The PointIterator Class

The `PointIterator` class is the primary class for accessing the point data in a LIDAR file. To create an iterator, call `PointSource::createIterator()`.

The `PointIterator` class only has one public method, `getNextPoints()`, which extracts points out of the point cloud. For each time you call `getNextPoints()` the function fills the given `PointData` buffer and returns the number of points that it extracted. When you have extracted all the points the function returns zero (0).

For an example, see "Chapter 4: Code Examples" on page 17.

Text Point Readers and Writers

The simplest and most flexible way of storing LiDAR data is using column delineated text (ASCII) files.

The `TXTPointReader` Class

This is a concrete implementation of the `PointSource` class for reading text files.

The `TXTPointWriter` Class

This is a concrete implementation of the `PointWriter` class for writing text files.

The `MG4PointReader` Class

The `MG4PointReader` class is the class that you will use to enable your application to read LiDAR-based MG4 files. `MG4PointReader` is a concrete implementation of the `PointSource` class (see "The `PointSource` Class" on page 9).

Opening an MG4 File

There are two methods of opening an MG4 file:

- `init()` with an `IO` object
- `init()` with a file name

The first is the preferred method; create a `FileIO` object for the file name and then pass the `FileIO` object to `MG4PointReader::init()` (see "The `IO` Classes" on page 13).

The second method takes a string for the file name.

NOTE: The string used in the second method is a native codepage string, which is much simpler to use for testing but can cause problems if you can't represent the file path in the codepage.

For an example of `MG4PointReader` class usage, see "Chapter 4: Code Examples" on page 17.

The Support Classes

The MrSID Decode SDK includes several supporting classes, however, you will probably only need to engage the `IO` classes and the `Bounds` class.

The `IO` Classes

The SDK provides an abstract mechanism for reading and writing data. These mechanism constitute the `IO` class.

The `IO` class provides methods for opening and closing the resource, reading and writing of byte arrays at a given offset in the resource, and getting and setting the resource size. This model is different from the UNIX `stdio` interfaces in that the file position is not stored in the `IO` object. It mim-

ics the `POSIX` `pread()` and `pwrite()` interfaces. This model ensures the thread safety of the `IO` subclasses, enabling you to read from the `IO` instances on multiple threads simultaneously.

The `FileIO` Class is a concrete implementation of the `IO` class for reading files from and writing files to disk.

The Bounds Class

The `Bounds` class defines a three-dimensional bounding box used to define regions of interest. This class has a one-dimensional interval for each of the X, Y and Z axes.

NOTE: The one-dimensional interval used by the `Bounds` class has member variables named `min` and `max` that may conflict with the `min()` and `max()` macros in `Windows.h`. To avoid this conflict, we undefined the `min()` and `max()` macros.

The Metadata Class

The `Metadata` class is a container for storing metadata about the point cloud. It is a key-value pair container that you can use to store strings, arrays of floating point values and raw binary data (BLOBs).

Each key-value pair has the following properties:

- key name
- description (optional)
- data type (string, reals, or BLOB, stored as a `MetadataDataType` enum)
- values and length

For a table of code examples included the SDK, see "Chapter 4: Code Examples" on page 17).

Known Metadata Key Names

The MrSID Decode SDK recognizes six fixed metadata key names and one key pattern, but will accept any name. The seven recognized names are listed in the following table.

Metadata Key Names

Name	Data Type	Description
<code>FileSourceID</code>	string	Identifies the source of the data.
<code>ProjectID</code>	string	Identifies the project that the data was acquired for.
<code>SystemID</code>	string	Identifies the hardware system or the method by which the file was made.
<code>GeneratingSoftware</code>	string	The software that created the file.
<code>FileCreationDate</code>	string	Date the file was created in the form yyyy-mm-dd
<code>PointRecordsByReturnCount</code>	array of reals	Contains an array of point

Name	Data Type	Description
PreCompressionPointCount	array of reals of length 1	counts per return Used to store the number of points that were in the input file before it was compressed/decimated.
LAS_BoundingBox	array of reals of length 6	Stores the bounding box of the original LAS file if the source was a LAS file.
<LAS VLR User ID>:- <Record ID>	BLOB	The method we use to store unrecognized variable length records (VLRs) from LAS files.

Chapter 4: Code Examples

The MrSID Decode SDK includes code samples that demonstrate the use of the SDK's different interfaces.

The following C++ (.cpp) files are located in your `examples/src` directory.

Code example files and what they demonstrate

<code>UserTutorial.cpp</code>	Opening MG4 files Using the <code>PointIterator</code> to access the point cloud Using <code>PointSource::read()</code> to access a fixed number of points
<code>DumpMG4Info.cpp</code>	Accessing the point cloud properties Displaying metadata
<code>DecodeMG4ToTXT.cpp</code>	Using a <code>PointWriter</code> class
<code>IterateOverPoints.cpp</code>	Using a <code>PointIterator</code> Accessing channel values from a <code>PointData</code> object
<code>support.cpp</code>	Using the <code>FileIO</code> class
<code>UserTest.cpp</code>	Enables you to add your own test code to explore the SDK

Below, we walk through the `UserTutorial.cpp` example.

The following code opens an MG4 file:

```
FileIO *file = FileIO::create();
file->init("data/Tetons_200k.sid", "r");
MG4PointReader *pointSource = MG4PointReader::create();
pointSource->init(file);
file->release();
```

Now that the file is initialized, you can access the properties of the point cloud using the following code:

```
PointSource::count_type numPoints = pointSource->getNumPoints();
size_t numChannels = pointSource->getNumChannels();
const PointInfo &pointInfo = pointSource->getPointInfo();

printf("Number of points: %lld\n", numPoints);
printf("Number of channels: %lu\n", numChannels);
for(size_t i = 0; i < numChannels; i += 1)
    printf("Channel %lu: %s\n", i, point-
Info.getChannel(i).getName());
```

You can use either of the following two methods to access the point cloud. In the first, we use the `PointIterator` mechanism.

```
PointData buffer;
// create buffers for all the channels 1000 samples long
buffer.init(pointInfo, 1000);
// create an iterator of the whole point cloud with all the chan-
nels
PointIterator *iter = pointSource->createIterator(pointSource-
>getBounds(),
                                                    1.0,
                                                    pointInfo,
                                                    NULL);

size_t count;
// walk the iterator
while((count = iter->getNextPoints(buffer)) != 0)
{
    // do some thing with this chunk of the point cloud.
}
iter->release();
```

The second method extracts a fixed number of points (10,000 in this case):

```
PointData buffer;
{
    // only decode X, Y, Z
    PointInfo pointInfo;
    pointInfo.init(3);
    pointInfo.getChannel(0).init(*pointSource->getChannel(CHANNEL_
NAME_X));
    pointInfo.getChannel(1).init(*pointSource->getChannel(CHANNEL_
NAME_Y));
    pointInfo.getChannel(2).init(*pointSource->getChannel(CHANNEL_
NAME_Z));
    buffer.init(pointInfo, 10000);
}

pointSource->read(Bounds::Huge(), buffer, NULL);
// do some thing with the points
```

Now we'll do a little housecleaning. When you're done with your point source, you should release it:

```
pointSource->release();
pointSource = NULL;
```

Chapter 5: MrSID Decode SDK Command Line Tools

The MrSID Decode SDK includes several command line tools you may find useful for decompressing MrSID Generation 4 (MG4) files or viewing information about MG4, LAS or text LiDAR files. These tools are located in the `bin` directory.

Decompressing MG4 Files

The MrSID Decode SDK includes a command line tool called `lidardecode`. Located in the `bin` directory, `lidardecode` enables you to decompress MG4 files to LAS or text files.

Usage

The only required parameters are `-inputFile` (or `-i`), which specifies the input file name, and `-outputFile` (or `-o`), which specifies the output file name.

If no output format (`-outputFormat` or `-of`) is specified, the file extension specified in the `-outputFile` parameter is used as the output format.

If no output format (`-outputFormat` or `-of`) is specified and no file extension is specified in the output file name, then `lidardecode` decodes the file to the default format (text) and appends the default suffix (`.txt`) to the output file name.

You may add other options and parameters as described in the table of switches below. The order of the switches in the syntax has no bearing on the output.

For examples of how to form a command, see "Examples" on page 20.

`lidardecode` Switches

<code>-inputFile (-i)</code>	string	(Required) Specifies name of input MG4 file.
<code>-outputFile (-o)</code>	string	(Required) Specifies name of output file. If no file extension is provided, default is to concatenate a format suffix to input file.
<code>-outputFormat (-of)</code>	string	Specifies output format. Acceptable values are TXT, LAS10, LAS11, LAS12. Default is TXT.
<code>-subsample (-s)</code>	unsigned integer	Tells <code>lidardecode</code> to subsample, taking every <i>n</i> -th point. <code>-s 2</code> selects one half the file, <code>-s 3</code> selects one third.
<code>-crop (-c)</code>	FLOAT0 FLOAT1 FLOAT2 FLOAT3 FLOAT4	Tells <code>lidardecode</code> to crop to the specified box (world coordinates: x-min, x-max, y-min, y-max, z-min, z-max). A value of <code>-inf</code> (for a minimum) or <code>+inf</code> (maximum) means do not crop in that direction.

FLOAT5

Continued >

<code>-offset (-ofs)</code>	FLOAT0 FLOAT1 FLOAT2	Specifies the offset from which the points will be specified (world coordinates: <code>x[0]</code> , <code>y[0]</code> , <code>z[0]</code>). Default is to use the origin of the bounding box.
<code>-outFields (-ofld)</code>	string	Tells <code>lidardecode</code> to include particular fields. By default <code>lidardecode</code> outputs all those supported by the output format and are in the input file. <code>x</code> - x point values <code>y</code> - y point values <code>z</code> - z point values <code>i</code> - intensity <code>r</code> - return number <code>n</code> - number of returns <code>d</code> - scan direction <code>e</code> - edge of flight line <code>a</code> - scan angle <code>c</code> - class id <code>p</code> - source id <code>u</code> - user data <code>t</code> - GPS time <code>R</code> - red <code>G</code> - green <code>B</code> - blue
<code>-scale (-sc)</code>	FLOAT0 FLOAT1 FLOAT2	Specifies the scale (or <i>precision</i>) factor (<code>x-scale</code> , <code>y-scale</code> , <code>z-scale</code>). Default is <code>0.001</code> , <code>0.001</code> , <code>0.001</code> .
<code>-h (-?)</code>		Displays a short usage message.
<code>-help</code>		Displays a detailed usage message.
<code>-version (-v)</code>		Displays version information.
<code>-verbose (-V)</code>		Tells <code>lidardecode</code> to display more verbose error messages.
<code>-credits</code>		Displays credits and copyrights.

Examples

The following command uses the minimum required parameters and decodes to a text file called "Exp_D2_1.txt".

```
lidardecode -i E:\Data\localTestImages\Exp_D2_1.sid -o Exp_D2_1.txt
```


The following command produces the same result as the previous one, but because the user wants to change the output file name, the `-o` parameter has been included and the text output is explicitly called for.

```
lidardecode -i E:\Data\localTestImages\Exp_D2_1.sid -o E:\Data-
\localTestImages\Exp_D2_2.txt
```

The following command decodes to a LAS file.

```
lidardecode -i E:\Data\localTestImages\Exp_D2_1.sid -o E:\Data-
\localTestImages\Exp_D2_1.las
```

The following command decodes to a text file called "Exp_D2_1.xyz" (any extension other than .las results in a text file) and limits the data in the file to four fields (GPS time, x, y and z).

```
lidardecode -i E:\Data\localTestImages\Exp_D2_1.sid -o E:\Data-
\localTestImages\Exp_D2_1.xyz -ofld txyz
```

Viewing File Information

The MrSID Decode SDK includes a command line tool called `lidarinfo`. Located in the `bin` directory, `lidarinfo` enables you to view the information in LAS or TXT files in text form.

Usage

The only required parameter for LAS files is `-inputFile` (or `-i`), which specifies the input file name. Text input files also require the `-parse` (or `-p`) parameter, which describes the order of the fields.

You may add other options and parameters as described in the table of switches below. The order of the switches in the syntax has no bearing on the output.

For examples of how to form a command, see "Example" on page 23.

lidarinfo Switches

<code>-inputFile (-i)</code>	string	(Required) Specifies the name of the input file.
<code>-parse (-p)</code>	string	(Required for text input) Parse format that describes the fields in a text input file. Valid values are: x - x point values y - y point values z - z point values i - intensity r - return number n - number of returns d - scan direction e - edge of flight line a - scan angle c - class id p - source id u - user data

t - GPS time
 R - red
 G - green
 B - blue
 s - skip this column

Example:

If you have five fields in the order GPS time, intensity, x, y and z and you only want the time and the point values, then specify `-parse tsxyz`, which skips the second (intensity) column and correctly labels the other four.

`-metadata (-m)`

Tells `lidarinfo` to display all metadata.

`-bounds (-b)`

Tells `lidarinfo` to determine the extents of the data by reading the data itself instead of reading min and max values reported in the header.

`-skipHeader (-skip) unsigned integer`

Tells `lidarinfo` to skip the first *n* lines of text input files.

`-returns (-r)`

Decodes the points and displays a histogram of the number of points per return value.

`-classification (-c)`

Decodes the points and displays a histogram of the number of points per classification.

`-h (-?)`

Displays a short usage message.

`-help`

Displays a detailed usage message.

`-version (-v)`

Displays version information.

`-verbose (-V)`

Tells `lidarinfo` to display more verbose error messages.

-credits

Displays credits and copyrights.

Example

The command

```
lidarinfo -i LakeRoosevelt_2.sid
```

returns the following information:

```
Basic LiDAR Info:
  Format:           MG4 4.0.0.1
  Number of Points: 3144893399
  Bounds Min:      408841.780000 5370276.770000 391.350000
  Bounds Max:      447234.600000 5422959.680000 1188.890000
  Scale:           0.001 0.001 0.001
  Offset:          408841.780000 5370276.770000 391.350000
  Supported Fields: GPSTime X Y Z Intensity ReturnNum NumReturns
  ClassId ScanDir ScanAngle UserData SourceId
  Spatial Reference: None
```


Appendix: Company and Product Information

This chapter contains information about LizardTech and its products as well as copyrights, trademarks and other information pertaining to this LizardTech software.

About LizardTech

LizardTech was founded in 1992 to build valuable business solutions from technologies created by the world's leading research organizations, including Los Alamos National Laboratory (LANL) and AT&T Labs.

LizardTech applies state-of-the-art technologies to the real-world challenges of managing, distributing, and accessing large, complex digital content such as aerial photographs and satellite imagery. LizardTech's software is installed on millions of desktops and integrated into a wide variety of platforms and applications.

LizardTech has offices in Seattle, London and Tokyo, and is part of Celartem Technology Inc. (Osaka Securities Exchange, Hercules:4330), a Japan-based technology company focused on storage and distribution technologies for digital images.

Other LizardTech Products

We at LizardTech are glad to have you creating products that support our software. We're confident that you will find the LizardTech LiDAR Decode SDK to be everything you need to build support for MrSID Generation 4 into your products. While you're "in the shop", explore LizardTech's other great products for compressing, managing and distributing geospatial imagery and LiDAR data.

For instant notification of new LizardTech product releases and patches add the following link to your favorite RSS feed reader:

<http://www.lizardtech.com/files/rss/updates.xml>

GeoViewer

Efficient Viewing and Exporting of MrSID and JPEG 2000 Layers

GeoViewer is LizardTech's free, standalone application for viewing geospatial imagery, vector overlays and LiDAR data. GeoViewer enables you to combine, view and export visual layers from varied sources, such as local repositories, Express Server catalogs, and WMS and JPIP servers. GeoViewer supports a wide range of input formats and exports to GeoTIFF, PNG and JPEG. It's the most efficient means of viewing MrSID and JPEG 2000 images.

For more information about GeoViewer visit <http://www.lizardtech.com/download/>.

ExpressView Browser Plug-in

Fast and Easy Viewing of Large Images

If you like the features in GeoViewer, consider ExpressView Browser Plug-in. ExpressView enables you to view, navigate and print MrSID and JPEG 2000 imagery in Internet Explorer or Firefox. Like GeoViewer, ExpressView enables you to save a portion of an image in a number of other image formats. ExpressView Browser Plug-in is quickly downloaded, easily installed, and free for individual use. It's the most convenient way to view MrSID and JPEG 2000 imagery over networks!

For more information about ExpressView Browser Plug-in visit <http://www.lizardtech.com/download/>.

GeoExpress

The Industry's Best Image Manipulation and Compression Software

With powerful tools for reprojecting, color balancing, mosaicking, and more, GeoExpress is the industry's choice for manipulating and compressing geospatial imagery to industry standard formats. You can also configure Express Server and Spatial Express directly from GeoExpress, which makes it the ideal command center for your storage and distribution workflows.

For more information about GeoExpress visit www.lizardtech.com/solutions/geo/.

LiDAR Compressor

LiDAR Data Meets the MrSID Format

LizardTech LiDAR Compressor enables you to turn giant point cloud data into efficient MrSID files that retain 100 percent of the raw data at just 25 percent of the file size (lossless compression). If you need even smaller files, LiDAR Compressor can reduce your LiDAR file sizes by 90 percent with no perceptible loss.* Unlike raw LAS or ASCII data, LiDAR files compressed to MrSID are easily managed resources you can extract derivatives from over and over again.

*On average, a LAS file compressed to MG4 at 10:1 using LiDAR Compressor has a RMSE (root mean square error) less than that of the sensor that captured the original data.

For more information about LiDAR Compressor visit www.lizardtech.com/solutions/lidar/.

Express Server

Image Delivery Software for Geospatial Workflows

LizardTech Express Server is the best solution for distributing imagery in MrSID or JPEG 2000 format. With Express Server, users on any device access imagery faster, even over low-bandwidth connections. Express Server is faster, more stable and easier to use than any other solution for delivering high-resolution raster imagery.

Express Server catalogs and manages the distribution of imagery, enabling you to integrate the world's best raster delivery technology with geospatial data stores anywhere. Express Server streams compressed images for the fastest possible delivery, whether conventionally to broadband devices and desktops or wirelessly to field personnel using low-bandwidth connections.

For more information about Express Server visit www.lizardtech.com/solutions/exp/.

Spatial Express

Database Storage and Retrieval Tools for Massive Geospatial Imagery

Spatial Express enables you to store, manage, and access massive image datasets efficiently using your Oracle Spatial database. Whether you want to maximize return on your database investment by storing raster image data hundreds of gigabytes in size, or reduce storage space requirements for that imagery by up to 95 percent while retaining image quality, or view the imagery in your geospatial applications, Spatial Express is the key link between your imagery, your database, and your users.

For more information about Spatial Express visit www.lizardtech.com/solutions/spatial/.

MrSID and Patents

This software is protected by patents and patents pending.

Index

A

Architecture and design 3

B

Buffer management classes 11

C

Class hierarchy 3

Code examples 17

Command line tools 19

 lidardecode 19

 lidarinfo 21

Contents 1

Copyrights, trademarks and credits ii

D

Decompressing MG4 files 19

F

Floating point quantization 4

I

Installation 6

L

Language bindings 2

 lidardecode 19

 lidarinfo 21

 LizardTech 25

M

Metadata class 14

MG4PointReader class 13

O

Other LizardTech products 25

P

PointInfo class 12

PointIterator class 12

PointSource class 9

PointWriter class 10

Programming and memory model 3

Q

Quantization 4

S

Sample code 17

SDK classes

 Buffer management 11

 metadata 14

 MG4PointReader 13

 PointIterator 12

 PointSource 9

 PointWriter 10

 support 13

 text point readers and writers 13

Specifying a region of interest 3

Support classes 13

SWIG bindings 2

System requirements 5

T

Technical support 6

 before you contact us... 7

Text point readers and writers 13

Thread safety 4

V

Viewing file information	21
--------------------------	----